# The five n = 131 canonicalizers inside a decomposition-style loop

2026-10-14

## The five n = 131 canonicalizers inside a decomposition-style loop

**Status (2026-10-14).**

- **Exactness: verified.** All five canonicalizers agree key for key, shift for shift and hit for hit inside the loop, on every key of every workload.
- **Deterministic counts: measured locally.** Ir and conditional-branch counts come from cachegrind. Its mispredict counts are a *model*.
- **Tier 1 timing: pending.** The paired, alternating GitHub-runner run under the frozen `protocol.json` (v2) starts with the push of this branch. Its receipts land in `results/ci/run-<id>-<attempt>/`.
- **This report makes no timing claim.** The local runs in `results/smoke*/` are script tests on an unisolated VM.
- **No curve or ECDLP result is involved.**

### Question

`canon-n131-20261014` measured five exact least-rotation canonicalizers for 131-bit normal-basis words in isolation. Their cachegrind Ir per uniform word were:

| arm | lift_scan | gap_scan | exhaustive | exhaustive_bf | refine_bf |
|---|---|---|---|---|---|
| Ir/word | 514 | 1,435 | 1,850 | 3,664 | 4,510 |

At n = 53 the isolated ranking did not survive the loop. The branch-free exhaustive vector scan beat variants A and B inside it (paper `papers/frobenius-orbits` §4/§7, `crypto:src/cryptanalysis/koblitz_fast.rs canon_many`). This experiment asks the n = 131 question. Inside a loop that has real independent work per key, what does each arm cost per key? Is the isolated ranking still the ranking?

### Field arithmetic: what `ecc2k130/` has, and what this uses

The brief said to look in `ecc2k130/` for a 131-bit normal-basis multiply before writing one. I found the following.

- **There is no normal-basis multiply as such.** But `ecc2k130/src/f131.h` has the walk's own path into its normal basis:
  - `mul` and `sqr`: polynomial-basis Karatsuba over PCLMULQDQ, with a direct reduction;
  - `fromPolynomial`: a linear network to the *permuted* type-II normal basis. Bit `i-1` holds the coefficient of `beta_i = gamma^i + gamma^-i`, where `gamma` has order 263.
- **So I did not need the pentanomial proxy multiply.** `src/lib.rs field::*` is a line-for-line Rust port of those three routines. `kat/f131_kat.cpp` builds the header itself and writes 400 known-answer vectors: the 131 unit vectors `z^k`, then random elements. The port matches the header on every vector, with the header built both with `ECC_HOST_CLMUL=1` and with `=0` (`results/kat_check.json`).
- **The permuted basis is not rotation-ordered.** In it, squaring is a permutation of the bits but not a rotation, so the canonicalizers cannot run on it directly.
  - **The fix.** Normal-basis coordinate `j` (the coefficient of `alpha^(2^j)`, `alpha = beta_1`) sits at permuted bit `fold(2^j mod 263) - 1`, where `fold(t) = t` if `t <= 131`, else `263 - t`. `to_cyclic` applies that fixed permutation with 33 nibble tables (12.7 KB).

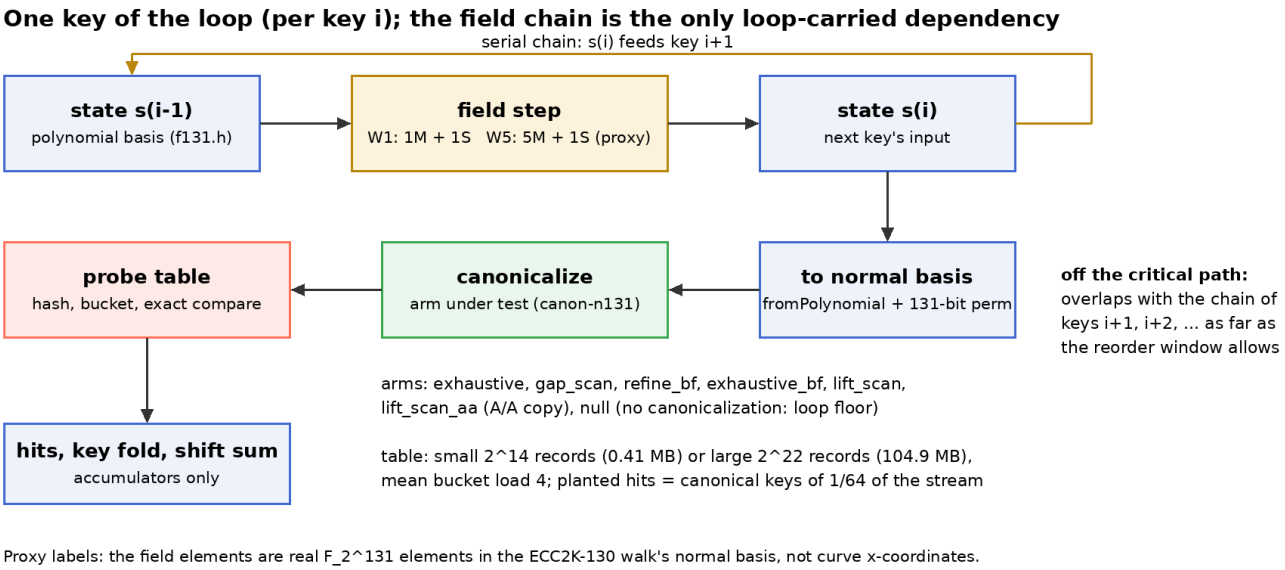



Figure 1: The loop and its dependency structure (`figures/loop.svg`).

  - **Checks.** The tests and the verifier confirm `to_cyclic(fromPolynomial(a^2)) = rotl(to_cyclic(fromPolynomial(a 1)` on 20,000 random elements and on all 2,097,152 stream words. They also confirm that the permuted basis alone fails this property.
  - **Cost.** This conversion is per-key work the real loop would also pay with this basis. Every arm pays it equally.
- **What is a proxy.** The per-key *field recurrence* stands in for a point operation. The elements are genuine elements of F_2^131 in the walk's normal basis, but they are **not x-coordinates of curve points**.
  - `W1`: one multiply and one squaring, `s' = s*g0 + s^2`.
  - `W5`: five multiplies and one squaring, about the field work of an affine addition with its share of a batched inversion.

## The loop

Per key `i`, a fused loop (`run_loop` in `src/lib.rs`) does four things:

1. **Field step.** `s(i) = step(s(i-1))` on the serial chain (W1 or W5).
2. **Conversion.** `x = to_cyclic(fromPolynomial(s(i)))`.
3. **Canonicalization.** `(k, shift) = canon(x)`, by the arm under test. The arms are canon-n131's own `G<131,64>` functions, used through a Cargo path dependency, not copied.
4. **Probe.** `hit = probe(k)`, in a hashed table:
   - a `u32` bucket directory over exact 24-byte records with mean bucket load 4;
   - every limb compared;
   - the same shape as the E-1 target stage at n = 83 (22-bit directory, 18-byte records there).

The hit count, a fold of `(k, shift)` and the shift sum are accumulated. The next state never depends on `k` or `hit`. So the conversion, the canonicalization and the probe of key `i` sit off the critical path. They can overlap with the field chain of later keys as far as the core's window allows. That overlap, and each arm's branches next to the probe's own branches, is what the loop adds to the isolated picture.

Each arm and work level is one `#[inline(never)]` symbol (`loop_<arm>_w1`, `loop_<arm>_w5`). Everything above is inlined into it.

**Finding: the A/A arm was merged away at first.** LLVM's function merging first folded `loop_lift_scan_aa_*` into `loop_lift_scan_*`: same address, so no A/A control. An empty `asm!` comment that names the arm now keeps them apart. It emits no instruction, and the two symbols now differ only in their addresses (checked with `objdump`).

| workload | per-key work | table | table bytes | stream |
|---|---|---|---|---|
| `w1-small` | W1 (1M+1S) | 2^14 records, 2^12 buckets | 409,604 | seed . . . 410 |
| `w1-large` | W1 | 2^22 records, 2^20 buckets | 104,857,604 | same stream as w1-small |
| `w5-small` | W5 (5M+1S) | 2^14 records, 2^12 buckets | 409,604 | seed . . . 450 |
| `w5-large` | W5 | 2^22 records, 2^20 buckets | 104,857,604 | same stream as w5-small |

**Planted hits.** The table holds two kinds of key:

- the exhaustive-reference canonical forms of the stream words at a pseudo-random 1/64 of positions: 8,291 for the W1 stream and 8,141 for the W5 stream, out of 524,288;
- filler canonical keys.

The planted positions are unpredictable to the branch predictor, and the exact hit set is known in advance.

## Exactness

All checks pass (`results/exactness.json`, `results/tests.txt`, `results/kat_check.json`).

| check | size | mismatches |
|---|---|---|
| field port vs `ecc2k130/src/f131.h` (mul, sqr, fromPolynomial), header built with clmul on and off | 400 vectors x 3 ops x 2 | 0 |
| `mul` vs bitwise schoolbook; `sqr(a)` vs `mul(a, a)` | 2,000 pairs | 0 |
| nibble-table permutation vs bitwise permutation | 20,000 random + 131 unit | 0 |
| Frobenius is `rotl` by 1 in the cyclic order (tests + every stream word) | 20,000 + 2,097,152 | 0 |
| in-loop `(key, shift, hit)`: each of exhaustive, gap_scan, refine_bf, exhaustive_bf, lift_scan, lift_scan_aa vs canon-n131 `exhaustive` applied to the independently generated stream, at every index | 4 workloads x 524,288 keys x 6 arms = 12,582,912 | 0 |
| the same, between arms (each vs the first) | same | 0 |
| canon-n131 `oracle` (a `Vec<bool>` oracle sharing no limb code) vs the reference | 4 x 20,000 | 0 |
| hit positions vs planted positions | 4 workloads x 6 arms | 0 differences: 8,291 (W1), 8,141 (W5) hits |
| timed entry point's summary vs record mode | 4 x 7 arms | 0 |

The `null` arm probes the raw word. It found 59 (W1) and 66 (W5) hits, the raw words that happen to be canonical already (about 1 in 131). By design it is excluded from the identity check.

## Deterministic counts (local, cachegrind)

**Instrument.** `perf` is not installed on this host, so every count is from `valgrind --tool=cachegrind --cache-sim=no --branch-sim=yes` (3.22.0).

- **What is counted.** Per symbol, a pass of 65,536 keys. The table build is outside the symbols.
- **What is exact and what is modelled.** Ir and Bc (conditional branches) are exact counts for this binary. **Bcm is cachegrind's simulated branch predictor, a model, not hardware mispredicts.**
- **Scope.** Ir and Bc do not depend on the table size, and the large-table rows agree to 0.1 per key (`results/counts.json`). The rows below are the small-table workloads.

| per key | null (loop floor) | exhaustive | gap_scan | refine_bf | exhaustive_bf | lift_scan |
|---|---|---|---|---|---|---|
| Ir, W1 | 1,041 | 3,924 | 2,801 | 5,833 | 6,387 | **1,560** |
| Ir, W5 | 1,932 | 4,811 | 3,690 | 6,720 | 7,276 | **2,439** |
| Ir over null (canonicalizer in the loop) | 0 | 2,883 | 1,760 | 4,792 | 5,346 | 519 |
| isolated canon-n131 Ir less its 6 Ir call overhead | | 1,844 | 1,429 | 4,504 | 3,658 | 508 |
| in-loop / isolated | | **1.56** | 1.23 | 1.06 | **1.46** | 1.02 |
| Bc, W1 | 47.0 | 437.0 | 319.4 | 176.9 | 176.9 | 74.1 |
| Bcm (model), W1 | 2.18 | 7.75 | 17.63 | 3.18 | 3.18 | 4.13 |
| Ir ratio to lift_scan, W1 (isolated) | | 2.52 (3.60) | 1.80 (2.79) | 3.74 (8.78) | 4.09 (7.13) | 1 |

**Findings (deterministic, this binary and toolchain).**

1. **Inlining changes what two arms compile to.** Inside the loop, `exhaustive` retires 1.56x its isolated Ir, and `exhaustive_bf` 1.46x.
   - **`exhaustive`.** Its inner loop is 22 instructions per rotation step (`objdump` of `loop_exhaustive_w1`). That is 130 x 22 = 2,860, against about 14 per step in the isolated `canon131_exhaustive`.
   - **The other three arms** stay within 1.23x (`gap_scan`), 1.06x (`refine_bf`) and 1.02x (`lift_scan`) of their isolated counts.
   - **Not diagnosed:** the cause. It could be register pressure or a different lowering of the limb compare. The isolated Ir of an inlined kernel is not a reliable prediction of its cost in a loop.
2. **The Ir ranking changes in the loop.** Isolated, `exhaustive_bf` (3,664) is below `refine_bf` (4,510). In the loop it is above (6,387 vs 5,833 per key, W1). The other three keep their order: `lift_scan < gap_scan < exhaustive`.
3. **`lift_scan` retires the fewest instructions in the loop on every workload.** In instructions, nothing closes its gap; the gap narrows only by the shared floor.
   - Its in-loop canonicalization costs 519 Ir per key over the null loop.
   - That is half the loop's own floor at W1 (1,041) and a quarter of it at W5 (1,932).
4. **Modelled mispredicts follow branch structure, not instruction count.**
   - `gap_scan` has the most modelled mispredicts: 15.4 per key over null, from its data-dependent scan over about 65 one bits.
   - The two branch-free arms add 1.0 per key over null. That is consistent with one mispredicted exit of their fixed 130-step loop per key, and their Bc is exactly null + 130.

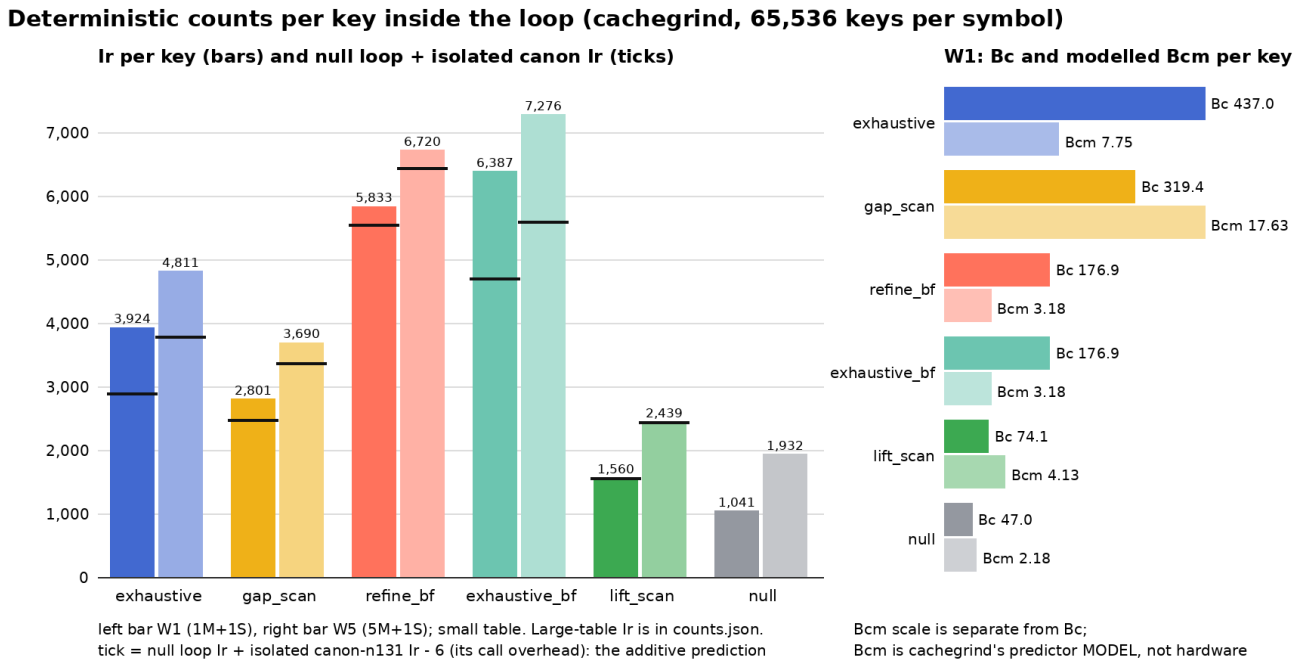



Figure 2: Ir, Bc and modelled Bcm per key (`figures/counts.svg`).

- **No hardware conclusion follows.** Whether mispredicts and overlap change the *time* ranking is the Tier 1 question below. The model is not hardware evidence.

## Tier 1 timing (pending; protocol frozen)

The workflow `.github/workflows/canon-n131-loop.yml` runs on every push to `claude/n131-canon-loop` and on `workflow_dispatch`. Its trigger paths exclude its own receipts (`results/ci/**`), this README, `figures/**` and `report.pdf`.

**Before timing.** It rebuilds, runs the tests, regenerates the f131.h known-answer vectors and reruns the full exactness verifier. Timing starts only when all of these pass.

**The timed run.** On one `ubuntu-24.04` hosted runner, pinned with `taskset -c 1`:

- **Rounds:** 41 rounds of 524,288 keys per pass.
- **Arms:** eight per workload per round:
  - the five arms;
  - `lift_scan_aa` (A/A, negative control);
  - `lift_scan_2pass` (positive control: two passes in one interval, a known 2x);
  - `null` (descriptive loop floor).
- **Order:** the arms rotate each round, and the rotation reverses every eight rounds.
- **Per pass:** wall ns, TSC ticks (reference cycles, not core cycles), thread CPU ns, voluntary and involuntary switches, the CPU id, and the output summary. The summary is checked against the verified reference. Mismatching rows are kept.

**Decision rule** (frozen; `analyze.py`):

- **Statistic.** For each workload and arm, `rho = t(arm) / t(lift_scan)` is paired by round. The summary is its geometric mean with a percentile bootstrap over rounds.
- **Primary family.** Exhaustive, gap_scan, refine_bf and exhaustive_bf vs lift_scan on four workloads, so m = 16. The CI is Bonferroni-adjusted to level 1 - 0.05/16.
- **Verdicts.**
  - "lift_scan faster in the loop" if the lower bound is above 1;
  - "`<arm>` faster than lift_scan" if the upper bound is below 1;

  - otherwise "not resolved".
- **Control gates.** A failed positive control (2pass lower bound not above 1.5) makes that workload "inconclusive". A nonnull A/A effect downgrades any contrast no larger than twice that effect.
- **Scope.** Every ratio is instance-scoped to the named runner. Tier 1, per AGENTS.md, does not prove host-wide exclusivity.

**Receipts.** `receipt.py` snapshots the allocation before and after the timed run: runner and image identity, the Azure instance metadata if reachable, `lscpu`, affinity, cgroup quota and throttling, `/proc/stat` steal, and load. The rows, the analysis and the timing figure (`make_figures.py` draws `figures/timing.svg` from the newest non-smoke analysis) are follow-up work once the run lands.

**Frozen predictions (protocol v1/v2; outcomes pending except P1)**

| | prediction | outcome |
|---|---|---|
| P1 | every exactness check passes | **held** (above) |
| P2 | lift_scan has the lowest median ns/key on all four workloads | pending (Tier 1) |
| P3 | each arm's in-loop ratio to lift_scan is below its isolated Ir ratio | pending; the in-loop *Ir* ratios are already below (table above) |
| P4 | exhaustive_bf and refine_bf stay slower than lift_scan in the loop | pending |
| P5 | A/A CI contains 1 on at least three of four workloads | pending |

## Deviations from the protocol

**Protocol v2** (commit `7ba2e4baf`) was adopted after one local dry run and before any Tier 1 timing. v1 was committed first (`babef1ad4`); its SHA-256 and the changed fields are in `protocol.json`, key `amendments`.

- **Why.** v1's positive control was "null faster than lift_scan". That is not a known effect: in the dry run, null did not separate from lift_scan on either large-table workload. Full overlap of lift_scan behind DRAM probe misses would produce exactly that.
- **The change.** v2 plants a known 2x effect on the same code path: `lift_scan_2pass`. It keeps `null` as a descriptive floor only.
- **Unchanged:** workloads, rounds, keys, the primary family, the reference arm and the decision rule.
- **Disclosure.** The dry run also showed every primary arm slower than lift_scan on all four workloads, by smaller ratios on the large tables. It was a 3-round, 16,384-key script test on the unisolated VM (`results/smoke_protocol_v1/`), so those numbers are not results.
- **Smoke run under v2.** `results/smoke/` is a script test only. Its `analysis.json` is labelled SMOKE.

## Claim classes (controlled-experiment §1)

| claim | class |
|---|---|
| the port equals `f131.h`; Frobenius is rotation in the cyclic order; all arms agree in the loop | exact computation (verified, reproducible) |
| Ir and Bc per key, the in-loop/isolated Ir ratios, the Ir ranking change | exact computation for this binary and toolchain |
| Bcm per key | model output (cachegrind's predictor), not a hardware measurement |
| any time per key, ratio, or "faster" | comparative: **not made here**; reserved for the Tier 1 run's decision rule |
| W1/W5 resemble a point operation | heuristic proxy, labelled |

**Scope.** This is a word-level loop with a proxy field recurrence. It shows nothing about ECDLP cost, curve points, or constant-time behaviour.

## Graphs checked (research-visuals)

- **Created:**
  - `figures/loop.svg`, a diagram;
  - `figures/counts.svg`, from `results/counts.json`.

  `make_figures.py` writes each SVG, and a small Pillow renderer of the same SVG subset rasterizes each PNG (ImageMagick has no SVG delegate on this host).
- **Pending:** `figures/timing.svg`. It is drawn once a non-smoke Tier 1 `analysis.json` exists.
- **No curve graph affected.** No curve, isogeny or volcano graph is involved. `experiments/volcano-ic/figures/` and `experiments/ic-candidate-catalog/curves.yaml` are unchanged.
- **Paper not updated.** `papers/frobenius-orbits` is not edited. The Ir findings above are for its owner's §4/§7 once the Tier 1 timing exists.

## Reproduce

```
export CARGO_TARGET_DIR=...        # optional shared build dir
python3 run.py                     # build (--locked --offline), tests, f131.h KAT, exactness,
                                   # cachegrind counts, SMOKE bench + analysis -> results/
taskset -c 1 $CARGO_TARGET_DIR/release/loop_bench protocol.json rows.jsonl   # the Tier 1 run
python3 analyze.py rows.jsonl analysis.json results/isolated_ir.json
python3 make_figures.py
pandoc README.md -o report.pdf --pdf-engine=pdflatex -V geometry:margin=1.8cm -V fontsize=10pt
```

**Build.** The crate's only dependency is the canon-n131 path crate. The `Cargo.lock` is committed, and `.cargo/config.toml` enables PCLMULQDQ. The build refuses to fall back without it.

**Receipts.** `results/build_receipt.json` holds the toolchain, the host, the protocol hash, and the SHA-256 of every source, dependency source (canon-n131 `lib.rs`, `f131.h`) and binary. `loop_counts`, whose binary produced `counts.json`, hashes the same as in the receipt.